



Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte/MG | Maio de 2026

Sistemas Operacionais
Professor Ítalo Cunha

Documentação TP2

Implementação proportional share CPU scheduler

Este documento detalha as modificações realizadas no código-fonte do sistema operacional xv6 para a implementação do escalonador por loteria

Alunos:

Daniel Vítor Rabelo Rodrigues | Matrícula: 2022043248

Ítalo Leal Lana Santos | Matrícula: 2024013893

Sarah Menks Sperber | Matrícula: 2023001824

1. Estrutura de dados para controle de tickets

Foram adicionados dois novos campos à struct proc definida em **kernel/proc.h**:

- **tickets**: armazena a quantidade de bilhetes de loteria atribuídos ao processo
- **ticks**: conta quantas vezes o processo foi escolhido pelo escalonador (É usado para depuração e para a interface pstat)

```
diff --git a/kernel/proc.h b/kernel/proc.h
--- a/kernel/proc.h
+++ b/kernel/proc.h
@@ -91,6 +91,8 @@ struct proc {
    int killed;    // If non-zero, have been killed
    int xstate;    // Exit status to be returned to parent's wait
    int pid;       // Process ID
+   int tickets;   // Lottery scheduling ticket count
+   int ticks;     // Times that the process won the lottery
```

O campo **tickets** armazena a quantidade de bilhetes de loteria atribuídos ao processo, determinando sua fatia proporcional de CPU. O campo **ticks** contabiliza quantas vezes o processo foi selecionado pelo escalonador, funcionando como contador de uso real de CPU, essencial tanto para depuração quanto para a interface **pstat**.

Ambos os campos são protegidos pelo spinlock próprio do processo (**p->lock**), evitando condições de corrida durante leituras e escritas concorrentes. Na inicialização do processo (**allocproc**), **tickets** é definido como 1 e **ticks** como 0. Processos criados via **fork()** herdam o valor de **tickets** do pai, preservando a distribuição da loteria entre gerações.

Observação de projeto: manter o total de tickets como variável global protegida por spinlock seria uma alternativa, mas optou-se por recalcular o total a cada rodada do escalonador percorrendo apenas processos **RUNNABLE**, evitando-se um lock extra de alta contenção no caminho crítico do scheduler.

2. Escolha do ticket vencedor

O gerador de números pseudoaleatórios (PRNG) e a lógica de seleção do processo vencedor foram implementados em **kernel/proc.c**:

```
diff --git a/kernel/proc.c b/kernel/proc.c
--- a/kernel/proc.c
+++ b/kernel/proc.c
@@ -68,6 +68,22 @@ cpuid()
+static uint64 random_next (void) {
+   rnd_state ^= rnd_state << 13;
+   rnd_state ^= rnd_state >> 7;
+   rnd_state ^= rnd_state << 17;
+   return rnd_state;
+}
+
+// Returns a random number between 0 and max, inclusive.
+uint64 random_at_most (uint64 max) {
+   if (max == 0) return 0;
+   return random_next() % (max + 1);
+}
```

No corpo do **scheduler()**, a seleção é feita em dois passos:

```
@@ -428,6 +449,11 @@ scheduler(void)
    struct cpu *c = mycpu();
    c->proc = 0;
+   int count = 0;
+   int winner = 0;
+   int total_tickets = 0;
    for(;;){
        // The most recent process to run may have had interrupts
        // turned off; enable them to avoid a deadlock if all
@@ -437,28 +463,61 @@ scheduler(void)
+   // Resets the count, winner and total_tickets for the next scheduling round
+   count = 0;
+   total_tickets = 0;
+   winner = 0;
+
+   // Calculate total tickets
+   for(p = proc; p < &proc[NPROC]; p++) {
+       acquire(&p->lock);
+       if(p->state == RUNNABLE) {
+           total_tickets += p->tickets;
+       }
+       release(&p->lock);
+   }
+
+   if(total_tickets > 0) {
+       winner = random_at_most(total_tickets - 1);
+   } else {
+       // nothing to run; stop running on this core until an interrupt.
+       asm volatile("wfi");
+       continue;
+   }
+
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == RUNNABLE) {
            // Switch to chosen process. It is the process's job
            // to release its lock and then reacquire it
            // before jumping back to us.
+           count += p->tickets;
+           if(count <= winner) {
+               release(&p->lock);
+               continue;
+           }
+           else{
+               p->state = RUNNING;
+               c->proc = p;
+               swtch(&c->context, &p->context);
+
+               // Se o processo foi executado, a quantidade de ticks aumenta em 1
+               p->ticks++;
+
+               // Process is done running for now.
+               // It should have changed its p->state before coming back.
+               c->proc = 0;
+               release(&p->lock);
+               break;
+           }
+       } else {
+           release(&p->lock);
+       }
    }
```

Implementamos um gerador **xorshift de 64 bits**, um PRNG clássico de baixo custo computacional e boa qualidade estatística, adequado para uso dentro do kernel. A sequência de shifts (**13** → **7** → **17**) é um conjunto padronizado com período máximo de **2⁶⁴-1**.

O algoritmo de seleção funciona da seguinte forma:

- (1) calcula-se o total de tickets de todos os processos **RUNNABLE**;
- (2) sorteia-se winner no intervalo [**0**, **total_tickets - 1**];
- (3) percorre-se a tabela acumulando **tickets**, o primeiro processo cuja soma acumulada supere winner é o escolhido. Após retornar de **swtch()**, incrementa-se **p->ticks**.

Se não houver processos **RUNNABLE**, a instrução **wfi** (wait-for-interrupt) coloca o núcleo em espera até o próximo evento, economizando energia.

3. Estrutura de pstat

A estrutura pstat foi definida no novo arquivo kernel/pstat.h:

```
diff --git a/kernel/pstat.h b/kernel/pstat.h
@@ -0,0 +1,13 @@
+#ifndef _PSTAT_H_
+#define _PSTAT_H_
+
+#include "param.h"
+
+struct pstat {
+ int inuse[NPROC]; // whether this slot of the process table is in use (1 or 0)
+ int tickets[NPROC]; // the number of tickets this process has
+ int pid[NPROC]; // the PID of each process
+ int ticks[NPROC]; // the number of ticks each process has accumulated
+};
+
+#endif // _PSTAT_H_
```

A syscall **sys_getpinfo()** percorre a tabela global de processos e preenche os quatro vetores de **pstat** para cada índice i:

- **inuse[i]**: 1 se o slot está ocupado (estado diferente de UNUSED), 0 caso contrário.
- **pid[i]**: identificador do processo.
- **tickets[i]**: número corrente de tickets.
- **ticks[i]**: número de vezes que o processo foi escalonado.

O lock de cada processo é adquirido individualmente durante a leitura para garantir uma captura consistente dos dados. Ao final, os dados são copiados para o espaço do usuário via **copyout()**. A syscall retorna 0 em caso de sucesso e -1 se o ponteiro passado for inválido ou nulo.

4. Outra Modificação: Herança de Tickets no kfork

Modificação realizada em **kernel/proc.c**, nas funções **allocproc()** e **kfork()**:

```

@@ -124,6 +140,8 @@ allocproc(void)
found:
    p->pid = allocpid();
    p->state = USED;
+   p->tickets = 1;
+   p->ticks = 0;
+
    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){

@@ -276,6 +294,9 @@ kfork(void)
}
np->sz = p->sz;
+ // 0 filho herda os tickets do pai
+ np->tickets = p->tickets;
+
    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);

```

Quando um processo chama **fork()**, o filho deve iniciar com a mesma quantidade de tickets que o pai. Sem essa herança, cada processo novo partiria com o padrão de 1 ticket, subvertendo a distribuição da loteria desejada pelo usuário.

A herança é implementada copiando o inteiro **p->tickets** antes que o filho se torne executável. Isso é consistente com o modelo clássico de lottery scheduling: tickets são uma propriedade herdável do processo. Por exemplo, uma tarefa intensiva em CPU que receba 100 tickets via **settickets()** e crie filhos via **fork** garante que os filhos competem com a mesma prioridade do pai.

A inicialização padrão em **allocproc()** define **tickets = 1** e **ticks = 0**, assegurando que todos os processos participem da loteria mesmo sem chamarem **settickets()** explicitamente.

5. Metodologia de testes

Os testes foram implementados no programa de usuário **tickettest.c**, composto por:

Teste 1: verificação do **getpinfo()**: chama **getpinfo()** e exibe o conteúdo completo de **pstat**, confirmando que todos os campos são preenchidos corretamente e que processos **UNUSED** têm campos zerados.

Teste 2: tickets esparsos: 6 processos com tickets {1, 4, 15, 64, 256, 660} (três ordens de grandeza de diferença), workload de 10^{11} iterações. Avalia se o scheduler prioriza corretamente processos com muito mais tickets.

Teste 3: poucos tickets: 4 processos com tickets {10, 20, 30, 40} (razão 1:2:3:4), workload de 10^{11} iterações. Caso mais próximo da relação 3:2:1 do enunciado — ideal para verificar proporcionalidade.

Teste 4: workload pequeno: 5 processos com tickets {1, 4, 16, 64, 256}, workload reduzido de 10^9 iterações. Avalia o comportamento com workloads que terminam rapidamente, causando distorção nas proporções.

Teste 5: tickets iguais por grupo: 9 processos em três grupos de 3 processos com tickets {1, 1, 1, 3, 3, 3, 9, 9, 9}. Verifica equidade intragrupo e proporcionalidade intergrupos.

Além disso, toda a suíte usertests foi executada (modo rápido e lento) para garantir que as modificações no escalonador não introduziram regressões. Todos os testes passaram, confirmando que o algoritmo lida corretamente com syscalls, alocação de memória, sistema de arquivos e sincronização entre processos.

6. Análise do gráfico

6.1 Referência: Teste 3 (razão aproximada de 1:2:3:4)

O teste mais representativo para a verificação gráfica da proporcionalidade é o Teste 3, onde quatro processos competem com tickets {10, 20, 30, 40}: razão 1:2:3:4, análoga à razão 1:2:3 pedida no enunciado (3:2:1 invertido). O total de tickets é 100 e o total de ticks observados foi 152.

Processo	Tickets	Ticks Esperados	Ticks Observados	Desvio (%)
A (PID 35)	10	15,2	27	+77,6%
B (PID 36)	20	30,4	38	+25,0%
C (PID 37)	30	45,6	39	-14,5%
D (PID 38)	40	60,8	48	-21,1%

O gráfico de barras (esperado $\times$ observado) evidencia que a tendência geral do scheduler é proporcional: processos com mais tickets acumulam mais ticks. Entretanto, observam-se desvios, o processo com 10 tickets recebeu quase o dobro do esperado (+77,6%), enquanto os de 30 e 40 tickets ficaram abaixo. Essa variância é inerente ao lottery scheduling: com uma carga de trabalho finita, a variância estatística do sorteio é significativa, especialmente para processos com poucos tickets. A tendência converge para a proporção esperada à medida que o número de sorteios aumenta.

6.2 Referência complementar: Teste 5 (grupos de tickets iguais)

O **Teste 5** valida dois aspectos simultaneamente: a proporcionalidade entre grupos (razão 1:3:9) e a equidade dentro de cada grupo (três processos com tickets iguais devem receber ticks semelhantes entre si). Total de tickets: 39. Total de ticks observados: 143.

Grupo	Tickets/proc.	Ticks Esperados (média)	Ticks Observados (média)	Desvio (%)
Grupo 1 ($\times 3$)	1	3,7	5,3	+43,2%
Grupo 2 ($\times 3$)	3	11,0	13,3	+20,9%
Grupo 3 ($\times 3$)	9	9	29,0	-12,1%

O Grupo 1 (1 ticket) recebeu em média 5,3 ticks (esperado: 3,7), e o Grupo 3 (9 tickets) recebeu 29,0 (esperado: 33,0). A tendência $1 < 3 < 9$ é preservada, confirmando que o scheduler distribui CPU proporcionalmente aos tickets.

A variância intragrupo é pequena (desvios entre os três processos de mesmo grupo), o que demonstra que processos com o mesmo número de tickets disputam a loteria em igualdade de condições.

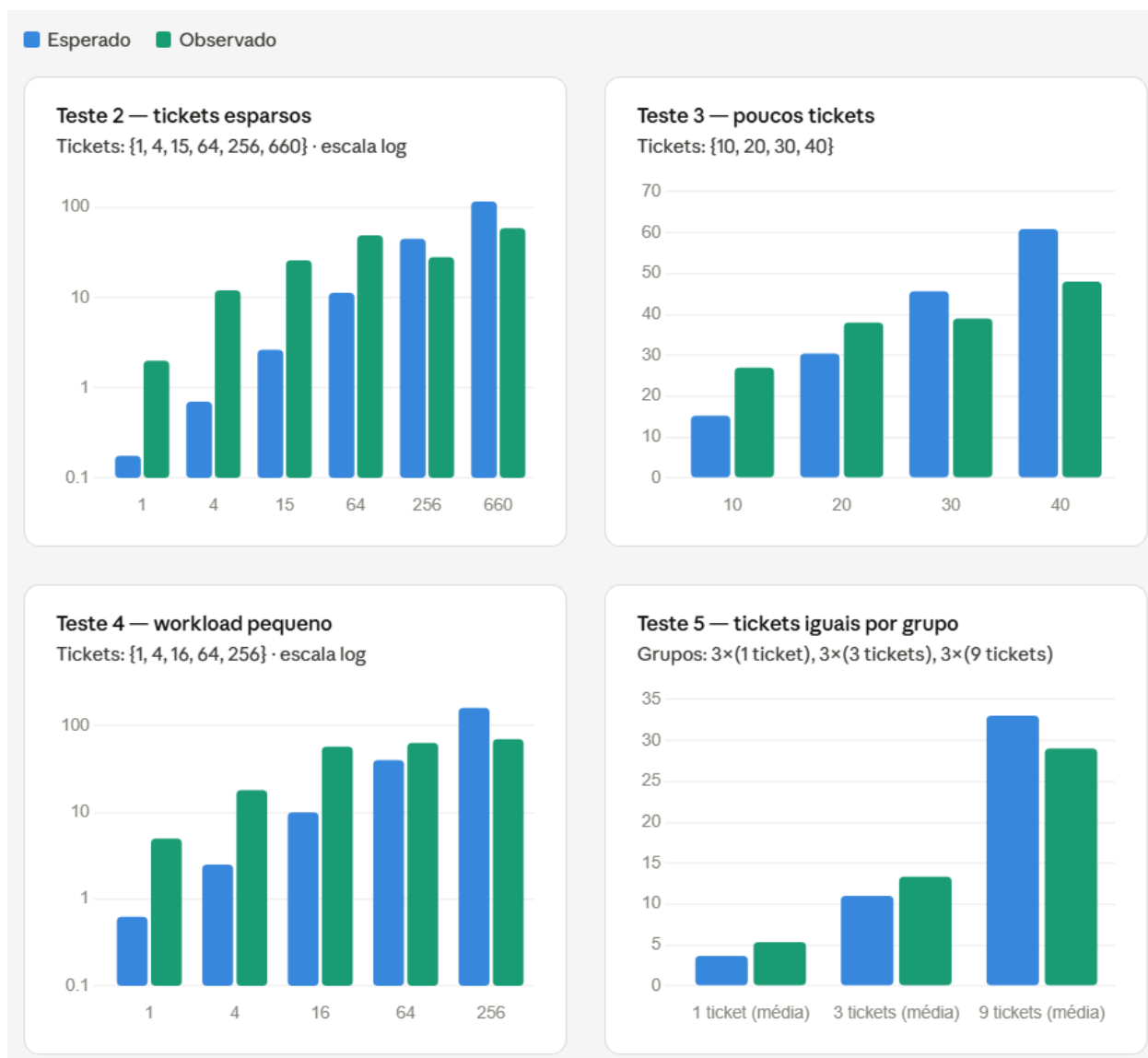
6.3 Interpretação dos desvios

Os desvios em relação ao ideal teórico têm três causas principais:

- **Variância estatística do sorteio:** Com workload finito, o número total de sorteios é limitado. A convergência para a proporção esperada é ruim. Desvios da ordem de 20% - 80% são esperados com dezenas de ticks.
- **Efeito de saída antecipada:** Quando processos com poucos tickets terminam antes dos outros, o pool de competidores muda. O total de tickets diminui, e os processos remanescentes recebem proporcionalmente mais ticks do que receberiam na presença dos que saíram.
- **Overhead de locking e context-switch:** O tempo de execução real inclui overhead do kernel (aquisição de locks, troca de contexto), que não é uniforme entre processos com cargas muito diferentes.

Em resumo, os resultados dos testes demonstram que o lottery scheduler funciona conforme esperado: a tendência proporcional é consistente e os desvios observados são estatisticamente explicáveis pelo tamanho finito da amostra

6.4 Gráficos



6.5 Observação: execução paralela com múltiplos cores

Foi observado que, ao executar os testes com **exatamente 3 processos** no xv6 rodando no QEMU com exatos **3 cores** habilitados, o sistema operacional escalona cada processo em um core distinto simultaneamente. Nesse cenário, os três processos executam em **paralelo real** e a loteria raramente (ou nunca) precisa escolher entre eles, cada core simplesmente roda o único processo RUNNABLE disponível para ele.

O efeito é visível nos dados da imagem abaixo: os processos com tickets **10, 40 e 100** acumularam ticks quase idênticos (44, 48 e 49, respectivamente), mesmo com a grande diferença nas proporções de tickets. Isso vai contra os resultados dos demais testes, onde a competição pela CPU é real e a proporcionalidade aparece. Essa observação reforça que o lottery scheduler só exibe seu comportamento proporcional quando **há mais processos RUNNABLE do que cores disponíveis** (condição necessária para que a disputa de tickets tenha efeito prático).

Iter	PID	Tickt	Ticks	InUse
0	1	1	35	1
1	2	1	13	1
2	3	1	16	1
3	4	10	44	1
4	5	40	48	1
5	6	100	49	1
6	0	0	0	0
7	0	0	0	0

7. Referências

DECODE. **Adding a system call in XV6 OS**. Acesso em: 25 maio. 2026. Disponível em: <<https://01siddharth.blogspot.com/2018/04/adding-system-call-in-xv6-os.html>>.

DECODE. **Implementing lottery scheduling on XV6**. Disponível em: Acesso em: 25 maio. 2026. <<https://01siddharth.blogspot.com/2018/04/implementing-lottery-scheduling-on-xv6.html>>.

WALDSPURGER, C. A.; WEIHL, W. E. **Lottery scheduling: flexible proportional-share resource management**. In: USENIX Symposium on Operating Systems Design and Implementation, 1994.

COX, R.; KAASHOEK, M. F.; MORRIS, R. **xv6: a simple, Unix-like teaching operating system**. MIT, 2022. Disponível em: <<https://pdos.csail.mit.edu/6.828/2022/xv6/book-riscv-rev3.pdf>>.